

Danh sách nhảy đoạn: một mở rộng của danh sách nhảy

Lý Ký Dương
Trường Trung học số 2 Thạch Gia Trang

Luận văn Trại đông 2009

Nguồn gốc: Danh sách nhảy đoạn: một mở rộng của danh sách nhảy

IOI China candidate team papers / 2009 / Li Jiyang / Segment Skip Lists

Abstract

Bài viết chủ yếu giới thiệu một mở rộng của tôi đối với danh sách nhảy *skip list* (còn gọi là danh sách nhảy, Skip Lists, SL, do William Pugh phát minh năm 1987): **danh sách nhảy đoạn** (*Segment Skip Lists, SSL*), cùng các chứng minh hiệu quả liên quan. Bài viết cũng phân tích ngắn gọn ưu thế của danh sách nhảy và các mở rộng của nó trong thi tin học.

Từ khóa. Danh sách nhảy; cây cân bằng; danh sách nhảy đoạn; cây đoạn.

Dẫn nhập

Phạm vi thảo luận trong bài này chỉ giới hạn ở danh sách nhảy ngẫu nhiên dựa trên kỳ vọng (Skip Lists, SL), không xét danh sách nhảy tất định kiểu 1-2-3 (1-2-3 Deterministic Skip List, 1-2-3 DSL).¹

Trong thi tin học, cấu trúc dữ liệu là một phần quan trọng. Các bài toán dạng này thường khảo sát năng lực duy trì và xử lý dữ liệu. Chúng hay yêu cầu ta duy trì một cấu trúc tuyến tính có thứ tự và thực hiện trên đó các thao tác như chèn, truy vấn. Những cách cài đặt thường dùng gồm cây cân bằng, danh sách liên kết chia khối, cây Fenwick, v.v.

Danh sách nhảy là một cấu trúc dữ liệu mới, hỗ trợ các thao tác từ điển như chèn, xóa và tìm kiếm trên bảng có thứ tự. Danh sách nhảy dựa trên ngẫu nhiên hóa và xác suất; độ phức tạp kỳ vọng của các thao tác chèn, xóa, tìm kiếm đều là $\mathcal{O}(\log N)$. Đồng thời, do bản chất vẫn là một bảng tuyến tính, nó có tính chất dễ viết và dễ mở rộng. Nó có thể thay thế cây cân bằng để cài đặt các thao tác từ điển. Hơn nữa, vì cấu trúc đơn giản và không cần thao tác xoay, danh sách nhảy hỗ trợ lưu các phần khác nhau trên các môi trường khác nhau và thực hiện thao tác song song.²

Tuy nhiên, danh sách nhảy cơ bản chỉ hỗ trợ các thao tác từ điển như chèn, xóa và tìm kiếm, chứ không hỗ trợ thao tác trên đoạn. Năng lực của danh sách nhảy không chỉ dừng ở đó: chỉ cần phân tích sâu hơn cấu trúc của nó, ta có thể rút ra từ danh sách nhảy một cấu trúc cây nhị phân. Từ đó thêm thông tin đoạn để thu được cây đoạn. Bài viết này sẽ nâng cấp các thao tác chèn và xóa để chúng có thể duy trì thông tin đoạn.

Danh sách nhảy sau khi mở rộng được gọi là danh sách nhảy đoạn. Nó có thể thay thế hoàn toàn nhiều loại cây cân bằng, hỗ trợ cả thao tác từ điển lẫn thao tác trên đoạn. Đồng thời nó vẫn giữ các đặc

¹Chú thích này tương ứng với chú thích mở đầu trong bản gốc.

²Nội dung liên quan xem *Concurrent Maintenance of Skip Lists* của William Pugh.

điểm vốn có: đơn giản, dễ dùng, và còn hỗ trợ những thao tác đặc biệt như xử lý tuần tự. Danh sách nhảy đoạn có thể trở thành một cấu trúc dữ liệu lý tưởng trong thi tin học.

Contents

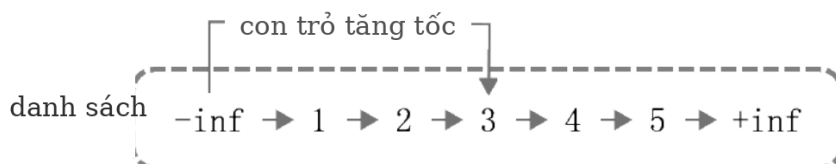
1	Giới thiệu danh sách nhảy	2
1.1	Từ danh sách liên kết đến danh sách nhảy	2
1.2	Mô tả hình thức cấu trúc danh sách nhảy	3
1.3	Các thao tác từ điển: tìm kiếm, chèn và xóa	4
2	Cấu trúc và thao tác cơ bản của danh sách nhảy đoạn	6
2.1	Trích xuất cây đoạn	6
2.2	Mô tả hình thức danh sách nhảy đoạn	8
2.3	Duy trì hai loại thông tin đoạn	9
3	Ưu thế và phân tích hiệu quả của danh sách nhảy	11
3.1	Phân tích hiệu quả của danh sách nhảy	11
3.2	Phân tích hiệu quả của danh sách nhảy đoạn	13
3.3	Ưu thế của danh sách nhảy	13
3.4	So sánh danh sách nhảy đoạn với cây đoạn và cây cân bằng	13

1 Giới thiệu danh sách nhảy

1.1 Từ danh sách liên kết đến danh sách nhảy

Danh sách liên kết, với tư cách một cấu trúc dữ liệu thông dụng và linh hoạt, thường được dùng để duy trì dữ liệu tuyến tính. Danh sách liên kết thực hiện chèn và xóa rất thuận tiện, nhưng chi phí tìm một số trong danh sách có thể rất lớn, với độ phức tạp thời gian $\mathcal{O}(N)$. Ngược lại, nếu duy trì một mảng có thứ tự, ta có thể tìm kiếm nhị phân trong thời gian $\mathcal{O}(\log N)$, nhưng để duy trì tính có thứ tự của mảng thì chèn và xóa lại có độ phức tạp lớn, cỡ $\mathcal{O}(N)$. Nếu có thể duy trì một danh sách liên kết có thứ tự đồng thời hỗ trợ tìm kiếm nhị phân, ta sẽ hoàn thành các thao tác chèn, tìm kiếm và xóa trên một tập dữ liệu quy mô n trong thời gian $\mathcal{O}(\log N)$.

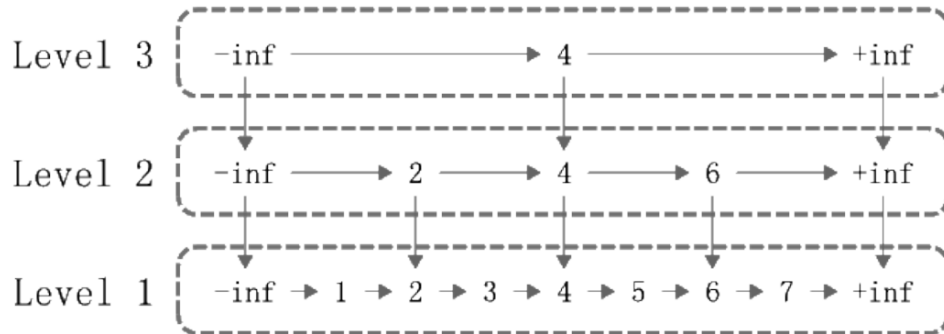
Trước hết hãy xét truy vấn trên một danh sách liên kết tĩnh. Một ý tưởng đơn giản là thêm cho một số nút trong danh sách liên kết một con trỏ phụ, gọi là con trỏ tăng tốc, để khi thích hợp ta có thể nhảy qua một số nút và tăng tốc tìm kiếm.



Hình 1.1. Danh sách liên kết có con trỏ tăng tốc: ngoài các cạnh nối tiếp thông thường còn có vài cạnh dài giúp bỏ qua nhiều nút.

Nhưng nếu giới hạn mỗi nút chỉ nối ra một con trỏ tăng tốc, dùng cấu trúc tương tự danh sách liên kết chia khối, độ phức tạp tốt nhất ta đạt được chỉ là $\mathcal{O}(\sqrt{n})$. Có thể thu được kết quả tốt hơn không? Ta nói

lỏng giới hạn “mỗi nút có một con trỏ tăng tốc”. Dựa trên phép tương tự với tìm kiếm nhị phân trong mảng tĩnh, ta có thể thu được cấu trúc sau.

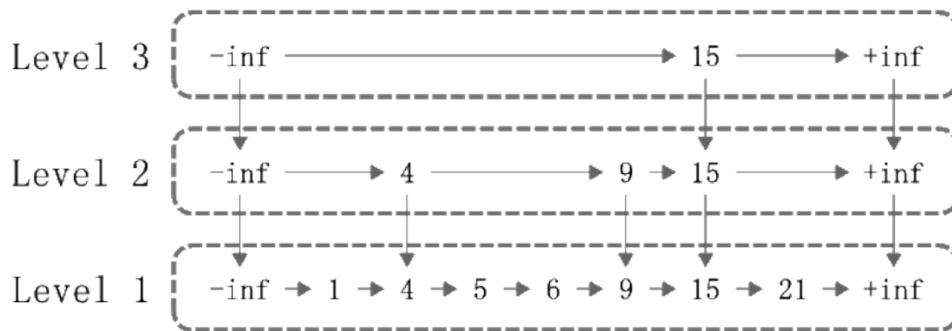


Hình 1.2. Một “danh sách nhảy tưởng tượng”: các con trỏ tăng tốc nối tới các nút giữa của những đoạn có thể xuất hiện trong quá trình tìm kiếm nhị phân.

Mỗi con trỏ tăng tốc nối ra từ một nút trỏ tới trung điểm của đoạn mà nút đó có thể làm đầu trái trong quá trình tìm kiếm nhị phân. Cấu trúc này có thể tìm kiếm dữ liệu trong thời gian $\mathcal{O}(\log N)$; quá trình tìm kiếm chính là tìm kiếm nhị phân, mỗi lần dùng con trỏ tăng tốc để tìm nút giữa của đoạn hiện tại rồi so sánh để xác định đoạn chính xác hơn và tiếp tục chia đôi.

Tuy nhiên, khi đối mặt với dữ liệu động, cấu trúc này rõ ràng không thể được duy trì với chi phí nhỏ trong quá trình chèn và xóa.

Ngẫu nhiên hóa thường được dùng để giảm độ phức tạp thời gian của thuật toán. Thêm tư tưởng ngẫu nhiên hóa vào hướng trên, ta thu được danh sách nhảy. Danh sách nhảy gồm Lev tầng danh sách liên kết và các con trỏ đi xuống. Khi chèn dữ liệu v , ta ngẫu nhiên sinh ra chiều cao h , rồi chèn các nút có khóa v vào các tầng từ 1 đến h ; với các nút từ tầng 2 đến tầng h , thêm con trỏ đi xuống trỏ tới nút tầng 1 có cùng khóa. h nút này tạo thành một khối (*block*) có chiều cao h .



Hình 1.3. Một danh sách nhảy gồm nhiều tầng danh sách liên kết; các bản sao của cùng một khóa được nối theo chiều dọc thành một khối.

Danh sách nhảy có cấu trúc như vậy có độ phức tạp kỳ vọng của tìm kiếm là $\mathcal{O}(\log N)$; xem phần 3.

1.2 Mô tả hình thức cấu trúc danh sách nhảy

Danh sách nhảy gồm nhiều danh sách liên kết $L_1, L_2, \dots, L_{maxlev}$ và các con trỏ đi xuống, thỏa mãn các điều kiện sau.

1. L_i là một danh sách liên kết, tăng đơn điệu từ trái sang phải, chứa hai phần tử đặc biệt $-\infty$ (nút bắt đầu) và $+\infty$ (nút kết thúc).

2. L_1 chứa mọi phần tử, L_i với $i \geq 2$ chứa một phần các phần tử, và với mọi $x \in L_i$ đều có $x \in L_j$ với mọi $1 \leq j < i \leq \maxlev$. Nói cách khác,

$$L_{\maxlev} \subseteq L_{\maxlev-1} \subseteq \dots \subseteq L_2 \subseteq L_1.$$

3. Với mỗi nút bất kỳ trong L_i ($i \geq 2$), có một con trỏ đi xuống trỏ tới nút trong L_{i-1} có cùng khóa.

Một số thuật ngữ dùng trong phần sau:

Cạnh ngang

Con trỏ trong các danh sách liên kết ở từng tầng.

Cạnh dọc

Con trỏ đi xuống.

Khối

Một thành phần liên thông nếu chỉ xét các cạnh dọc; số nút trong thành phần đó gọi là chiều cao của khối.

Nút trước, nút sau

Nút đứng ngay trước hoặc ngay sau một nút trong danh sách liên kết nơi nó thuộc về.

Nút đáy

Nút nằm ở tầng thứ 1.

Đường tìm kiếm Path(v)

Dãy nút đi qua khi tìm khóa v trong danh sách nhảy.

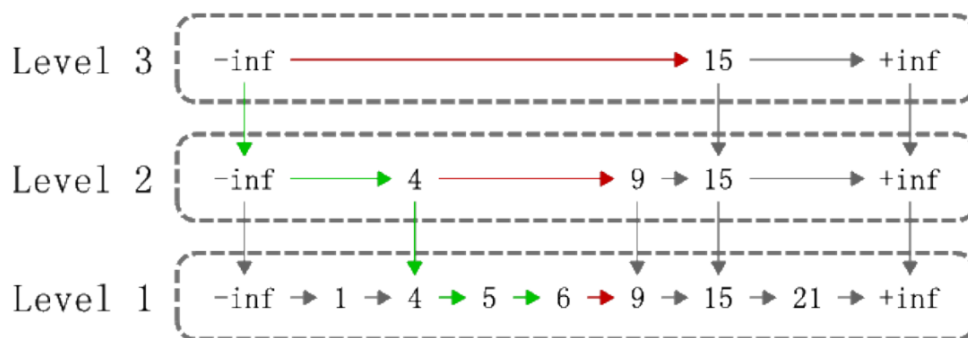
Đường tìm kiếm ngược Rev-Path(v)

Dãy thu được bằng cách đảo ngược Path(v).

1.3 Các thao tác từ điển: tìm kiếm, chèn và xóa

Tìm kiếm

Quá trình tìm kiếm chính là quá trình xây dựng Path(v). Con trỏ p bắt đầu từ nút đầu $-\infty$ ở tầng trên cùng L_h . Mỗi lần, nếu khóa của nút sau nút hiện tại nhỏ hơn hoặc bằng giá trị cần tìm, ta dịch p sang phải đến nút sau; ngược lại, ta đi theo cạnh dọc xuống tầng dưới. Khi không thể di chuyển nữa (đang ở nút tầng đáy và khóa của nút sau lớn hơn giá trị cần tìm), nút mà p trỏ tới chính là giá trị lớn nhất trong dữ liệu không vượt quá giá trị tìm kiếm.



Hình 1.4. Một quá trình tìm kiếm: đường màu xanh là đường tìm kiếm, các cạnh màu đỏ là những cạnh đã thử so sánh nhưng không đi qua.

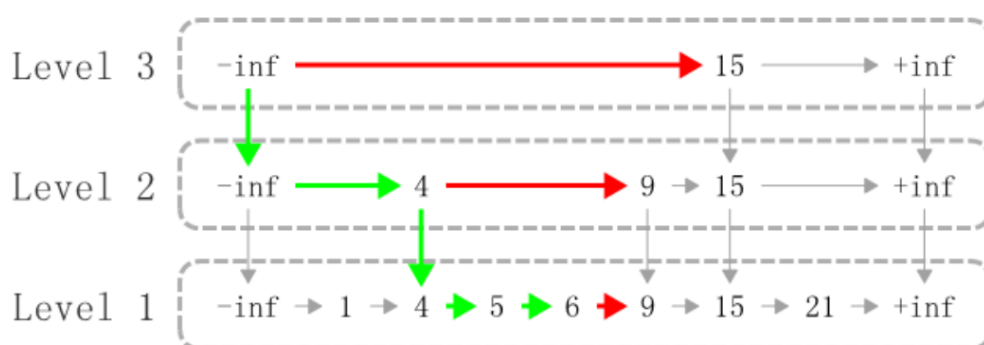
Chèn

Trước hết ngẫu nhiên hóa chiều cao của nút cần chèn. Cách sinh như sau. Mỗi nút được chèn ít nhất vào tầng cơ sở, nên chiều cao ban đầu ít nhất là 0. Trong vòng lặp, mỗi lần có xác suất p làm chiều cao tăng thêm 1, và xác suất $1 - p$ thoát vòng lặp, từ đó thu được chiều cao ngẫu nhiên cần dùng. Để lập trình thuận tiện, có thể đặt chiều cao tối đa $maxlev$ và cố định duy trì $maxlev$ danh sách liên kết. Nếu chiều cao sinh ra lớn hơn $maxlev$, ta gán nó bằng $maxlev$.

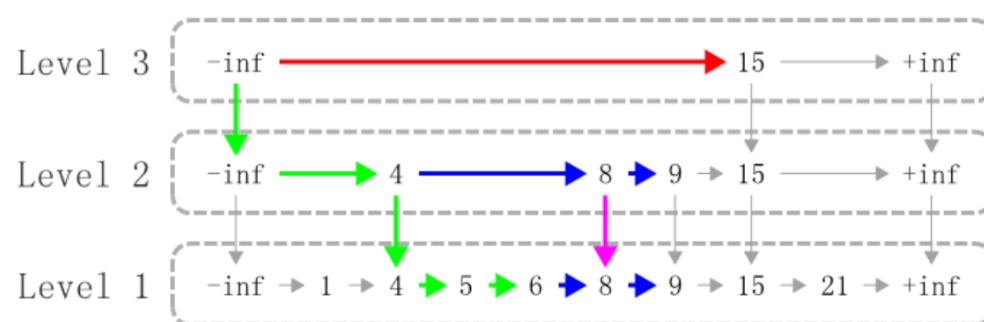
Giá trị của xác suất p thường được chọn là $1/4$; phân tích cụ thể để ở phần 3. Mã giả:

```
Function random_height;
  p := 1 / 4;
  h := 0;
  while random() <= p do h := h + 1;
  // random() tra ve mot gia tri trong [0, 1)
  if h > maxlev then h := maxlev;
  return h;
```

Sau đó chèn nút vào các tầng 1, ..., $random_height$ của danh sách nhảy.



Hình 1.5.1. Trạng thái trước khi chèn: các cạnh đỏ chỉ vị trí trên từng tầng nơi nút mới sẽ được chèn vào.

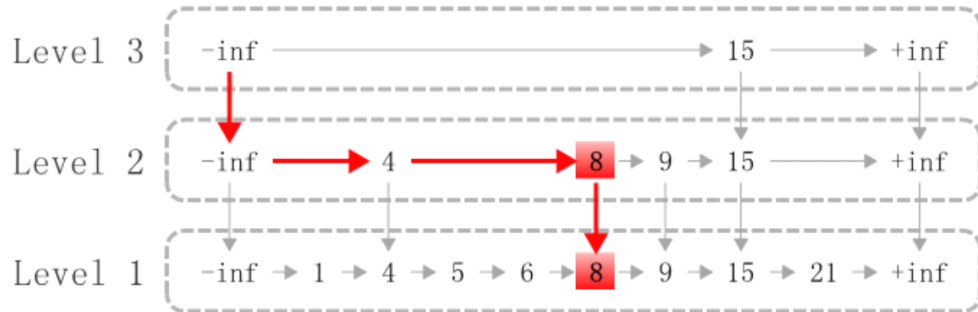


Hình 1.5.2. Trạng thái sau khi chèn: nút mới xuất hiện trên các tầng đã chọn ngẫu nhiên, đồng thời các cạnh dọc nối các bản sao của nó.

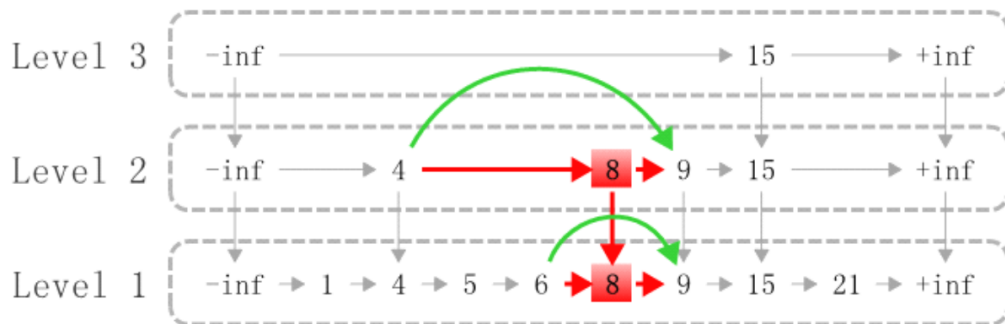
Khi cài đặt, trước tiên có thể tìm giá trị cần chèn để nhận được $Path(v)$, rồi chèn nút mới sau các nút của những tầng 1, ..., $random_height$ được thăm cuối cùng trong $Path(v)$. Nói cách khác, ở mỗi tầng trong các tầng đó, chèn nút mới sau nút được thăm cuối cùng.

Xóa

Xóa tương tự chèn. Trước tiên tìm giá trị cần xóa để nhận được $\text{Path}(v)$, rồi xóa khỏi danh sách nhảy các nút có khóa v .



Hình 1.6.1. Trước tiên tìm các nút cần xóa.



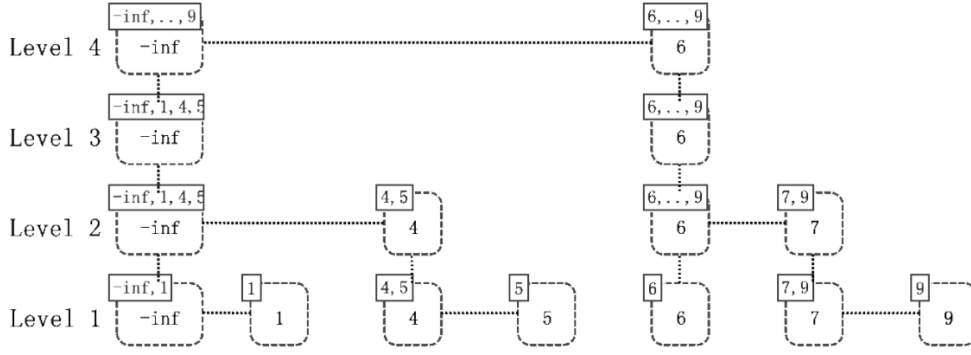
Hình 1.6.2. Xóa các nút và cạnh màu đỏ, rồi nối lại các cạnh màu xanh để khôi phục các danh sách liên kết từng tầng.

Mã liên quan trong bản gốc được đặt ở phụ lục `SkipList.Pas`.

2 Cấu trúc và thao tác cơ bản của danh sách nhảy đoạn

2.1 Trích xuất cây đoạn

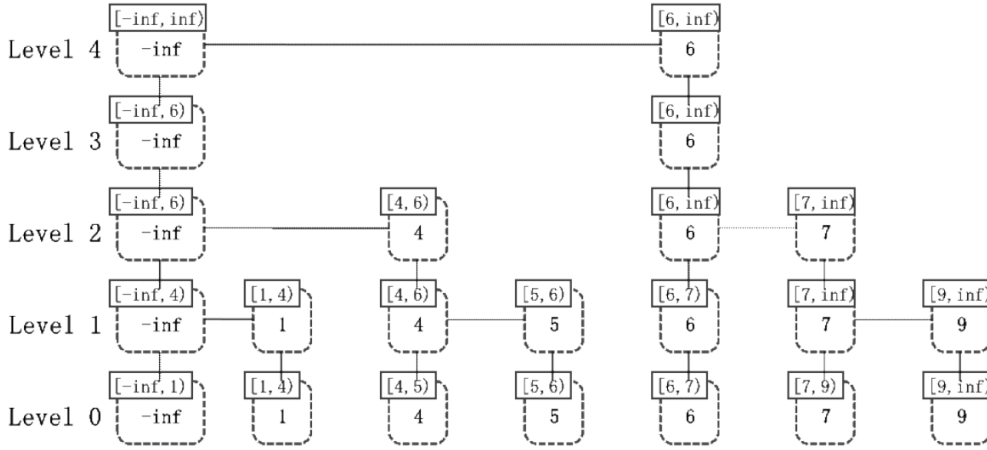
Mục này trích xuất cây đoạn từ cấu trúc danh sách nhảy, rồi từ đó thu được danh sách nhảy đoạn để duy trì thông tin đoạn. Với một danh sách nhảy cho trước:



Hình 2.3.1. Tiền thân của danh sách nhảy đoạn: các điểm rời rạc được đánh dấu dọc theo các đường tìm kiếm.

Ở đây ta đã có thể thấy bóng dáng của cây đoạn.

Tiến thêm một bước, thay các điểm rời rạc được đánh dấu bằng các đoạn liên tục. Đồng thời thêm tầng thứ 0 bên dưới tầng thứ 1, cũng chứa tất cả khóa, và thêm cho mọi nút tầng thứ 1 con trỏ đi xuống tới nút tương ứng ở tầng thứ 0. Ta thu được:



Hình 2.3.2. Phiên bản cuối cùng của danh sách nhảy đoạn: một cây đoạn ẩn trong danh sách nhảy, có thêm tầng 0 để biểu diễn các lá.

Đến đây, ta đã thu được một cây đoạn ẩn trong danh sách nhảy. Tôi gọi nó là **danh sách nhảy đoạn** (*Segment Skip List*, SSL).

2.2 Mô tả hình thức danh sách nhảy đoạn

Danh sách nhảy đoạn gồm nhiều danh sách liên kết $L_0, L_1, L_2, \dots, L_{maxlev}$ và các con trỏ đi xuống, thỏa mãn các điều kiện sau.

1. L_i là một danh sách liên kết, tăng đơn điệu từ trái sang phải, chứa hai phần tử đặc biệt $-\infty$ (nút bắt đầu) và $+\infty$ (nút kết thúc).
2. L_0 và L_1 chứa mọi phần tử, L_i với $i \geq 2$ chứa một phần các phần tử, và với mọi $x \in L_i$ đều có $x \in L_j$ với mọi $0 \leq j < i \leq maxlev$. Tức là

$$L_{maxlev} \subseteq L_{maxlev-1} \subseteq \dots \subseteq L_2 \subseteq L_1 = L_0.$$

3. Với mỗi nút bất kỳ trong L_i ($i \geq 1$), có một con trỏ đi xuống trỏ tới nút trong L_{i-1} có cùng khóa.

4. Mỗi nút có khóa nhỏ hơn $+\infty$ tương ứng với một đoạn nửa kín nửa hở. Biên trái của đoạn là khóa của nút đó. Giả sử trong quá trình tìm nút này, cạnh dọc cuối cùng đi qua là $u \rightarrow v$; khi đó biên phải của đoạn là khóa của nút sau u . Giá trị này không cần lưu thêm, vì trong mỗi quá trình tìm kiếm có thể thu được động.

Ngoài việc định nghĩa các nút tầng 0 là nút đáy, các thuật ngữ đã nêu ở phần trước về cơ bản không đổi.

Định nghĩa như vậy nhất định cho ra một cây đoạn, trong đó nút gốc biểu diễn đoạn $[-\infty, +\infty)$. Điều này rất dễ chứng minh; quá trình chứng minh được lược bỏ ở đây.

2.3 Duy trì hai loại thông tin đoạn

Tiếp theo là tìm cách dùng cây đoạn ẩn trong danh sách nhảy để duy trì thông tin đoạn. Trước hết chia thông tin đoạn thành hai loại:

Thông tin DP (DP_i)

Thông tin của một đoạn được lưu khi thực hiện quy hoạch động trên cây, chẳng hạn số điểm có khóa nằm trong đoạn, giá trị lớn nhất, giá trị nhỏ nhất. Đặc trưng là mỗi nút đều lưu thông tin này. Thông tin có thể được tính từ thông tin của hai con của nút, thường trong thời gian $\mathcal{O}(1)$.

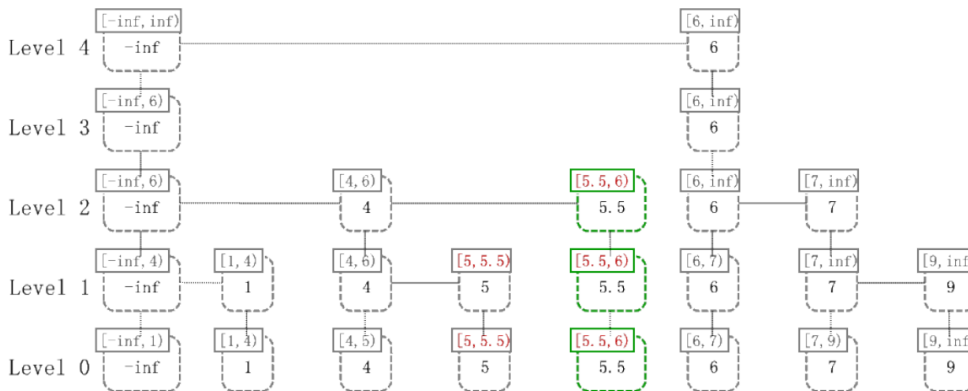
Thông tin đoạn (SEG_i)

Thông tin được lưu trong cây đoạn, chẳng hạn trạng thái tô màu của một đoạn. Đặc trưng là không phải mọi nút đều lưu thông tin này. Khi thực hiện thao tác đoạn, nếu đoạn thao tác chứa đoạn tương ứng với một nút không phải lá, có thể lưu thông tin tại nút không phải lá đó để giảm độ phức tạp. Thông tin cũng có thể được đẩy xuống hai nút con; ví dụ màu của một đoạn có thể được phân phối cho hai con.

Hai loại thông tin này có đặc điểm khác nhau, vì vậy phương pháp thao tác cũng khác nhau. Sau đây giới thiệu cách danh sách nhảy đoạn duy trì hai loại thông tin này khi chèn và xóa.

Chèn

Mặc dù sau khi chèn nút, cấu trúc cây đoạn có thể thay đổi, nhưng như hình dưới cho thấy, số nút có đoạn tương ứng bị thay đổi chỉ ở quy mô $\mathcal{O}(\log N)$.



Hình 2.4.1. Sự thay đổi các đoạn do nút SSL biểu diễn trong quá trình chèn. Các nút viền xanh là nút mới thêm; các đoạn chữ đỏ là những đoạn bị thay đổi. Ngoài một phần của $Path(v)$ và $Path(v - \varepsilon)$, đoạn tương ứng với các nút khác không đổi.

Vì vậy số nút cần cập nhật thông tin cũng chỉ có quy mô $\mathcal{O}(\log N)$.

Thứ tự thao tác khi chèn v :

1. Dọc theo $\text{Path}(v)$, đẩy thông tin SEG_i xuống, và thông qua quá trình này thu được thông tin SEG_i của nút v .
2. Chèn khối có khóa v , đồng thời đặt giá trị SEG_i của nút đáy bằng giá trị SEG_i vừa tìm được.
3. Theo thứ tự $\text{Rev-Path}(v)$, cập nhật thông tin DP_i của các nút trên đường.
4. Theo thứ tự $\text{Rev-Path}(v - \varepsilon)$, cập nhật thông tin DP_i của các nút trên đường.

Chỉ cần chứng minh hai kết luận sau là đủ để đảm bảo tính đúng đắn:

1. Mọi nút thuộc $\text{Path}(v)$ đều biểu diễn đoạn chứa v .
2. Sau khi chèn v , chỉ các nút trong $\text{Path}(v)$ và $\text{Path}(v - \varepsilon)$ có phạm vi đoạn biểu diễn bị thay đổi.

Chứng minh như sau. Với kết luận thứ nhất, mỗi lần đi tới một nút thì khóa của nút đó nhất định nhỏ hơn hoặc bằng mục tiêu tìm kiếm. Do đó mọi điểm trong $\text{Path}(v)$ có khóa nhỏ hơn hoặc bằng v , tức biên trái đóng của đoạn do mỗi điểm biểu diễn nhất định không vượt quá v . Với biên phải của đoạn, nó nhất định xuất hiện tại một lần so sánh nào đó: sau khi phát hiện v nhỏ hơn giá trị này, ta không đi sang phải nữa mà đi xuống. Vì thế biên phải mở của đoạn nhất định lớn hơn v .

Với kết luận thứ hai, trước hết khóa của từng điểm không đổi, nên biên trái của mọi đoạn không đổi. Vì vậy chỉ cần xét biên phải. Theo định nghĩa, biên phải là khóa của nút sau nút xuất phát của cạnh dọc cuối cùng trong quá trình tìm kiếm. Với các nút nằm ngoài $\text{Path}(v)$ và $\text{Path}(v - \varepsilon)$, con trở ngang nổi ra từ nút xuất phát của cạnh dọc đó không thay đổi. Tổng hợp lại, sau khi chèn v , chỉ các nút trong $\text{Path}(v)$ và $\text{Path}(v - \varepsilon)$ có phạm vi đoạn biểu diễn thay đổi.

Những nút cần thay đổi thông tin đoạn chỉ là các nút có phạm vi đoạn bị đổi và các nút có đoạn tương ứng chứa v , tức chỉ các điểm trên $\text{Path}(v)$ và $\text{Path}(v - \varepsilon)$.

Xóa

Xóa tương tự chèn. Đại thể quá trình như sau:

1. Dọn các thông tin SEG_i sẽ bị che phủ. Dọc theo $\text{Path}(v)$, sau khi lần đầu gặp $\text{SEG}_i \neq \text{NULL}$, đặt SEG_i của mọi nút phía sau trên đường này thành rỗng. Sau đó lặp lại một lần dọc theo $\text{Path}(v - \varepsilon)$.
2. Nếu trong $\text{Path}(v)$, thông tin SEG_i của nút có khóa v không hoàn toàn rỗng, thì v là đầu trái của một đoạn tô màu nào đó và không thể xóa. Tiếp tục xét trong $\text{Path}(v - \varepsilon)$ các nút có số tầng không lớn hơn chiều cao của khối có khóa v : nếu thông tin SEG_i của chúng đều rỗng, thì v là đầu phải của một đoạn nào đó và không thể xóa. Với những thông tin SEG_i có thể gộp được, trước hết nên gộp ra giá trị trên $\text{Path}(v)$ và $\text{Path}(v - \varepsilon)$. Nếu không rơi vào các trường hợp trên thì tiếp tục.
3. Xóa khối có khóa v .
4. Dọc theo $\text{Rev-Path}(v)$, cập nhật thông tin DP_i .

Để rút gọn độ dài, ở đây tôi viết chung thao tác duy trì hai loại thông tin. Nhưng trong ứng dụng thực tế thường chỉ dùng một loại thao tác. Phụ lục bản gốc cung cấp riêng hai chương trình làm tham khảo cho hai loại thông tin: $\text{SEG}_{i\text{SSL.Pas}}$ và $\text{DP}_{i\text{SSL.Pas}}$.

3 Ưu thế và phân tích hiệu quả của danh sách nhảy

3.1 Phân tích hiệu quả của danh sách nhảy

Độ phức tạp thời gian kỳ vọng của tìm kiếm, chèn và xóa

Trước hết giả sử $maxlev \rightarrow +\infty$. Ta suy ra độ phức tạp kỳ vọng của tìm kiếm, tức độ dài kỳ vọng của đường tìm kiếm.³

Gọi $f(N)$ là độ dài kỳ vọng của đường tìm kiếm trong danh sách nhảy có N nút. Nó gồm ba phần:

1. Độ dài kỳ vọng của đường tìm kiếm trong danh sách nhảy tạo bởi các tầng từ 1 đến $maxlev$. Phần này tương đương với độ dài kỳ vọng của một danh sách nhảy có quy mô kỳ vọng $\mathcal{O}(pN)$.
2. Một con trỏ đi từ tầng thứ hai xuống tầng thứ nhất. Trong danh sách nhảy đoạn, đây là con trỏ đi từ tầng thứ nhất xuống tầng thứ không.
3. Độ dài đường đi sang phải ở tầng thứ nhất. Trong danh sách nhảy đoạn, đây là độ dài đi sang phải ở tầng thứ không. Mỗi lần có thể đi sang phải với xác suất $1 - p$, ký hiệu là q .

Do đó

$$\begin{aligned} f(N) &= f(pN) + 1 + q + q^2 + q^3 + q^4 + q^5 + \dots \\ &= f(pN) + \frac{1}{p}. \end{aligned}$$

Giải hàm truy hồi

$$f(N) = f(pN) + \frac{1}{p}$$

ta được

$$f(N) = \log_a N, \quad a = \left(\frac{1}{p}\right)^p.$$

Độ phức tạp thời gian của chèn và xóa gồm một phần liên quan tới độ dài đường tìm kiếm và một phần liên quan tới chiều cao, tức

$$\mathcal{O}(\log_a N + max_height) = \mathcal{O}(\log_a N),$$

trong đó max_height là chiều cao của danh sách nhảy. Khi $maxlev$ là một hằng số, không tiến tới vô cùng, ta có $max_height = maxlev$. Do bị giới hạn bởi $maxlev$, độ phức tạp thời gian trong tình huống này không dễ phân tích, nhưng trong thực tế không gây suy biến, trừ khi đặt $maxlev$ quá nhỏ. Nó làm việc lập trình đơn giản hơn, vì có thể cố định số tầng của danh sách nhảy là $maxlev$; cách này được khuyến nghị. Cách chọn $maxlev$ xem bên dưới.

Độ phức tạp không gian kỳ vọng

Vẫn giả sử $maxlev \rightarrow +\infty$.

1. Kỳ vọng chiều cao của một nút đơn lẻ:

$$h_x = 1 + p + p^2 + p^3 + p^4 + \dots = \frac{1}{1-p}.$$

³Mục này đưa ra một suy diễn ngắn gọn, trực quan nhưng chưa hoàn toàn nghiêm ngặt. Chứng minh chặt chẽ xem *Skip Lists: A Probabilistic Alternative to Balanced Trees* của William Pugh.

2. Độ phức tạp không gian kỳ vọng:

$$\text{số nút} = N \times h_x = \frac{N}{1-p}.$$

3. Phân tích chiều cao lớn nhất:

$$\Pr(h_{\max} \leq m) = (1 - p^{m-1})^N.$$

4. Khi maxlev là một hằng số, không tiến tới vô cùng, kỳ vọng chiều cao của nút không vượt quá giá trị phân tích ở trên, và độ phức tạp không gian cũng không vượt quá giá trị thu được ở trên.

Xác suất p

Với

$$a = \left(\frac{1}{p}\right)^p,$$

ta có

$$f(N) = \log_a N = \frac{\log_2 N}{\log_2 ((1/p)^p)}.$$

Khi $p = 1/e$, hệ số là nhỏ nhất:

$$f(N) = \frac{\log_2 N}{\log_2 (e^{1/e})} \approx 1.88416939 \log_2 N.$$

Với $p = 1/2$ và $p = 1/4$, hệ số đều là

$$f(N) = \frac{\log_2 N}{\log_2 (2^{1/2})} = \frac{\log_2 N}{\log_2 (4^{1/4})} = 2 \log_2 N.$$

$p = 1/e$ có thời gian ngắn hơn $p = 1/2$ và $p = 1/4$ một chút, nhưng khó cài đặt hơn, nên thông thường dùng $p = 1/2$ hoặc $p = 1/4$. Vì lý do không gian, $p = 1/4$ được dùng phổ biến hơn.

Hệ số không gian là

$$b = \frac{1}{1-p}.$$

Từ đó biết được khi $p = 1/4$, lượng không gian dùng ít hơn $p = 1/2$; xấp xỉ bằng 2/3 so với khi $p = 1/2$. Vì vậy giá trị p thường dùng nhất là 1/4.

Giới hạn số tầng maxlev

Khi không có giới hạn tầng cao nhất, tức $\text{maxlev} \rightarrow +\infty$, mới là danh sách nhảy chuẩn. Nhưng trong ứng dụng thực tế, để cài đặt đơn giản, ta thường đặt một hằng số như vậy. Sau khi đưa vào maxlev , độ phức tạp thời gian không dễ phân tích, nhưng hiệu quả thực nghiệm tốt. Xem phần phân tích chiều cao lớn nhất: có thể chọn một maxlev phù hợp. Ví dụ với $p = 1/4$, khi $N = 10^6$ chọn $\text{maxlev} = 16$, ta có

$$\Pr(h_{\max} \leq \text{maxlev}) = (1 - p^{\text{maxlev}-1})^N \approx 0.999069111,$$

là một lựa chọn không tệ.

3.2 Phân tích hiệu quả của danh sách nhảy đoạn

Các thao tác của danh sách nhảy đoạn về cơ bản giống danh sách nhảy, hiệu quả cũng phụ thuộc vào độ dài kỳ vọng của đường tìm kiếm. Độ phức tạp thời gian vẫn là $\mathcal{O}(\log N)$, chỉ có hằng số lớn hơn.

Về độ phức tạp không gian, do nó nhiều hơn danh sách nhảy thường một tầng, có thể ghi thô là

$$\mathcal{O}\left(\frac{2-p}{1-p}N\right).$$

Thoạt nhìn khá lớn, nhưng thực ra khi $p = 1/4$ thì chỉ là

$$\mathcal{O}\left(\frac{7}{3}N\right),$$

chỉ lớn hơn cây đoạn một chút. Do mỗi nút cần ghi thông tin đoạn, kích thước bản thân nút cũng tăng. Nhưng độ phức tạp không gian vẫn là $\mathcal{O}(N)$.

3.3 Ưu thế của danh sách nhảy

- Bản chất của danh sách nhảy là danh sách liên kết, thao tác đơn giản, không cần thao tác xoay. Danh sách nhảy ngắn hơn nhiều cây cân bằng thông thường. Quan trọng hơn, các thao tác của nó dễ suy nghĩ, không cần kỹ thuật viết đặc biệt để rút ngắn mã.
- Hỗ trợ thao tác song song. Danh sách nhảy không cần xoay, cấu trúc không sinh ra thay đổi lớn, có thể lưu trên các môi trường khác nhau và hỗ trợ thao tác song song. Xem *Concurrent Maintenance of Skip Lists* của William Pugh.
- Chỉ cần thay đổi giá trị p là có thể điều chỉnh hiệu quả thời gian và không gian.
- Danh sách nhảy nguyên thủy, chỉ hỗ trợ thao tác từ điển, không cần lưu bất kỳ thông tin phụ nào để duy trì cân bằng mà vẫn đạt độ phức tạp thời gian tốt. Không có lãng phí không gian.
- Có thể thực hiện các thao tác đặc biệt như thao tác tuần tự, tức xử lý các nút theo thứ tự khóa.

3.4 So sánh danh sách nhảy đoạn với cây đoạn và cây cân bằng

Cây đoạn và cây cân bằng là các cấu trúc dữ liệu thường dùng trong thi tin học, rất nhiều bài toán có thể dùng chúng. Danh sách nhảy đoạn được giới thiệu trong bài này dùng ngẫu nhiên hóa để thu được một cấu trúc tương đối cân bằng, đồng thời có thể hỗ trợ mọi chức năng của cây đoạn và cây cân bằng. So với cây đoạn, nó có đặc điểm là có thể xử lý thông tin động và hỗ trợ truy vấn trực tuyến. So với cây cân bằng, nó có đặc điểm là dễ viết và mã ngắn gọn.

Tóm lại, danh sách nhảy và mở rộng của nó, danh sách nhảy đoạn, là các cấu trúc dữ liệu xuất sắc. Chúng có thể trở thành một công cụ sắc bén cho các thí sinh tham gia thi tin học.

Tài liệu tham khảo

1. William Pugh, *Skip Lists: A Probabilistic Alternative to Balanced Trees*.
2. William Pugh, *Concurrent Maintenance of Skip Lists*.
3. Wei Ran, luận văn Trại đông 2005, *Làm hiệu quả thuật toán “nhảy lên”! – Bàn sơ về các thao tác liên quan và ứng dụng của danh sách nhảy*.